



Analisis Proses Serangan DoS dan ARP Spoofing antar Perangkat ESP32 pada WSN Bertopologi Star

^{[1]*}Dede Kurniawan, ^[2]Uray Ristian, ^[3]Hafiz Muhandi

^{[1][2][3]}Program Studi Rekayasa Sistem Komputer, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Tanjungpura, Pontianak, Indonesia

e-mail: ^{[1]*}h1051211106@student.untan.ac.id, ^[2]eristian@siskom.untan.ac.id, ^[3]hm@siskom.untan.ac.id

Email Penulis Korespondensi: h1051211106@student.untan.ac.id

Submitted: 24/09/2025; Accepted: 01/10/2025; Published: 06/04/2026

Abstrak

Perangkat Internet of Things (IoT) seperti ESP32, meskipun populer karena murah dan fitur Wi-Fi, memiliki keterbatasan sumber daya yang membuatnya rentan terhadap serangan siber. Penelitian ini menganalisis bagaimana serangan dapat diimplementasikan menggunakan 1 perangkat ESP32 dan dampak terhadap 1 perangkat ESP32 dari dua jenis serangan umum, yaitu Denial of Service (DoS) melalui ICMP Flood dan Address Resolution Protocol (ARP) Spoofing, dalam lingkungan Wireless Sensor Network (WSN) bertopologi star. Dalam penelitian ini, satu perangkat ESP32 diprogram sebagai penyerang untuk menyerang salah satu dari tiga perangkat ESP32 client lain yang terhubung ke sebuah server lokal melalui protokol WebSocket. Tujuan penelitian ini adalah untuk menganalisis pengimplementasian serangan, dampak performa perangkat target, dan dampak terhadap metrik Quality of Service (QoS) seperti latency, jitter, throughput, dan packet loss, serta kinerja perangkat (CPU load dan memori). Hasil penelitian menunjukkan bahwa serangan ARP Spoofing yang dilancarkan oleh satu perangkat ESP32 efektif dan dapat digunakan sebagai serangan DoS karena mampu memutus koneksi client target dari server dengan tingkat packet loss mencapai hampir 100%. Sebaliknya, serangan ICMP Flood dari satu perangkat ESP32 penyerang tidak berhasil mematikan perangkat target tetapi secara konsisten meningkatkan beban CPU-nya mencapai 17,63%. Penelitian ini membuktikan bahwa perangkat berdaya rendah seperti ESP32 dapat digunakan sebagai platform serangan yang efektif dalam jaringan lokal, menyoroti pentingnya keamanan pada lapisan jaringan bahkan di antara perangkat sejenis.

Kata kunci: ESP32, Denial of Service, ARP Spoofing, Keamanan IoT, WSN.

1. PENDAHULUAN

Dalam beberapa tahun terakhir, ESP32 telah menjadi mikrokontroler yang sangat populer dalam aplikasi Jaringan *Internet of Things* (IoT)[1]. Fitur Wi-Fi terintegrasi dengan biaya yang rendah memudahkan konektivitas antar perangkat, namun di sisi lain membuka tantangan keamanan yang signifikan dalam transmisi data[2]. Perangkat IoT pada umumnya, termasuk ESP32, sering kali rentan terhadap serangan siber karena keterbatasan sumber daya komputasi seperti prosesor, kapasitas memori, dan fitur keamanan bawaan.

Serangan umum pada lingkungan IoT mencakup *Denial of Service* (DoS), *Man-in-the-Middle* (MITM) yang seringkali melibatkan *ARP Spoofing*, dan akses tidak sah[3]. Serangan DoS bertujuan untuk mengganggu ketersediaan layanan dengan membanjiri perangkat target hingga koneksinya terputus atau responsnya melambat[4]. Sementara itu, *ARP Spoofing* memungkinkan penyerang untuk memanipulasi tabel ARP dalam jaringan lokal untuk mencegah, mengubah, atau menghentikan komunikasi antar perangkat[5]. Penelitian sebelumnya telah mengkaji analisis forensik terhadap serangan *ARP spoofing*[6] dan

tantangan keamanan umum di IoT[2], namun analisis mendalam mengenai kapabilitas perangkat berdaya rendah seperti ESP32 untuk menjadi platform serangan masih terbatas.

Penelitian ini bertujuan untuk menganalisis dan menunjukkan bahwa satu perangkat berbiaya rendah seperti ESP32 dapat digunakan sebagai alat penyerang yang efektif dalam jaringan *Wireless Sensor Network* (WSN)[7]. Dengan mensimulasikan serangan DoS (via *ICMP Echo Request Flood*) dan ARP *Spoofing* dari satu ESP32 ke ESP32 lainnya dalam topologi *star*, penelitian ini akan mengukur dampak serangan secara kuantitatif terhadap metrik *Quality of Service* (QoS) meliputi *latency*, *throughput*, *jitter*, dan *packet loss* serta kinerja perangkat target (CPU load dan memori)[8].

2. METODOLOGI PENELITIAN

Metodologi penelitian ini dirancang untuk mensimulasikan dan menganalisis serangan DoS dan ARP *spoofing* dalam lingkungan WSN terkontrol. Tahapan penelitian meliputi studi literatur, analisis kebutuhan, perancangan sistem, implementasi, pengujian, dan penarikan kesimpulan.



Gambar 1. Diagram Alir Penelitian

2.1 Studi Literatur

Studi literatur dilakukan untuk memahami konsep dasar mengenai serangan *Denial of Service* (DoS) dan *Address Resolution Protocol* (ARP) *Spoofing*. Referensi diambil dari jurnal, buku, dan sumber artikel yang relevan. Informasi yang diperoleh dari studi literatur ini menjadi dasar pengetahuan yang penting untuk mendukung perumusan ide dan konsep dalam penelitian. Fokus utamanya adalah Cara kerja serangan DoS dan ARP *spoofing* menggunakan ESP32, cara menggunakan ESP32 yang dapat dimanfaatkan untuk menyerang, dan dampak dari serangan DoS dan ARP *spoofing* terhadap ESP32.

2.2 Analisis Kebutuhan

Analisis kebutuhan terkait sistem yang akan dibangun sesuai dengan rancangan kebutuhan. Hal ini berkaitan dengan perangkat keras dan perangkat lunak pada penelitian.

2.2.1 Kebutuhan Perangkat Keras

Adapun kebutuhan perangkat keras dalam membangun sistem yaitu:

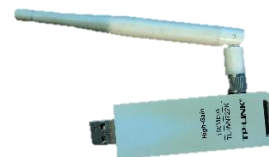
1. ESP32 sebagai mikrokontroler pada sistem ini yang mendukung Wi-Fi. Perangkat ini sering digunakan sebagai perangkat prototipe untuk berbagai aplikasi IoT. ESP32 digunakan pada penelitian ini sebagai *client* yang mengirim data metrik performa ke server untuk mensimulasikan bagaimana ESP32 digunakan pada lingkungan IoT. Selain itu, perangkat ESP32 juga digunakan sebagai penyerang salah satu dari tiga *client* yang berada dalam jaringan.



Gambar 2. ESP32

Gambar 2 merupakan ESP32 yang digunakan pada penelitian ini, jumlah perangkat yang digunakan sebanyak 4 unit.

2. Server sebagai penyedia atau penerima data metrik dari ESP32. *Server* berjalan menggunakan *virtual machine*. Penelitian ini menggunakan laptop Acer Aspire 3 A315-41-R66A dengan spesifikasi processor AMD Ryzen 3 2200U, RAM 16 GB dengan penyimpanan harddisk 1 TB dan SSD 256 GB.
3. Wi-Fi *Adapter* TP-Link dengan seri TL-WN722N digunakan sebagai perangkat *network capture*. Untuk adapter akan berjalan di mode *monitor*, agar pergerakan paket antara client dan penyerang dapat di analisis.



Gambar 3. TP-Link TLWN722N

Gambar 3 merupakan perangkat dari TP-Link yang akan digunakan sebagai penangkap paket data ketika client dan server mengirim data menggunakan protokol WebSocket.

4. *Access Point* sebagai penghubung ESP32 penyerang, ESP32 *client*, dan *server*.

Perangkat yang akan dijadikan *access point* menggunakan produk dari TOTOLINK dengan seri N300RT. Perangkat ini memiliki support metode enkripsi WPA/WPA2 untuk mencegah access yang tidak berizin.



Gambar 4. TOTOLINK N300RT

Gambar 4 merupakan perangkat *access point*. Perangkat ini sebagai penghubung *server* dan *client* Pada *password* menggunakan jenis enkripsi WPA2.

2.2.2 Kebutuhan Perangkat Lunak

Adapun kebutuhan perangkat lunak dalam membangun sistem yaitu:

1. Visual Studio Code sebagai aplikasi pendukung pemrograman pada sistem ini.
2. PlatformIO sebagai pendukung pemrograman perangkat keras yang diinstal ke dalam aplikasi Visual Studio Code. PlatformIO adalah ekosistem perangkat lunak bersifat *open-source* yang dirancang secara profesional untuk pengembangan sistem tertanam[9].
3. ESP-IDF sebagai *framework* resmi yang disediakan oleh Espressif System untuk produk *System on Chip* (SoC) mereka, terutama seri ESP32. Framework ini dirancang untuk pengembangan aplikasi IoT[10].
4. WebSocket adalah protokol jaringan yang memungkinkan komunikasi dua arah antara *client* yang menjalankan *untrusted code* dalam lingkungan yang terkontrol dengan *host* jarak jauh yang telah menyetujui untuk berkomunikasi dengan kode atau *handshake*[11]
5. Virtual Machine Workstation adalah salah satu aplikasi *virtual machine*. *Virtual machine* adalah sebuah bentuk emulasi perangkat lunak dari suatu sistem komputer yang memiliki CPU, RAM, media penyimpanan, dan antar muka jaringan sendiri, namun semuanya diwujudkan dalam bentuk perangkat

lunak yang berjalan di atas satu mesin fisik utama atau disebut dengan *host*[12].

6. Kali Linux adalah sebuah sistem operasi distribusi Linux berbasis Debian yang bersifat *open-source*. Sistem operasi ini dapat dijalankan di berbagai jenis perangkat keras dan tersedia secara gratis, sehingga dapat diakses baik oleh para profesional keamanan informasi maupun para pegiat atau hobi di bidang siber[13].
7. Airmon-ng adalah perangkat lunak yang digunakan untuk menghidupkan mode *monitor* pada *wireless interfaces* atau sebaliknya biasanya disebut *managed mode*.
8. Aircrack-ng merupakan perangkat lunak dari aircrack-ng dengan aircrack-ng dapat mendecrypt paket *capture* dengan format WEP/WPA/WPA2[14].
9. Tcpdump merupakan aplikasi penangkap lalu lintas jaringan berbasis *command-line interface*[15]. Aplikasi ini dijalankan bersamaan didalam *server*.
10. Node.js adalah *runtime environment* JavaScript yang bersifat *open-source* dan *cross-platform*[16].
11. Wireshark merupakan aplikasi penangkap lalu lintas jaringan berbasis *graphical user interface*[17]. Aplikasi ini digunakan setelah sistem diuji akan menghasilkan data *packet capture*.
12. SQLite adalah sebuah library perangkat lunak yang mengimplementasikan mesin basis data SQL yang bersifat mandiri (*self contained*), tanpa *server*, tidak memerlukan konfigurasi, dan transaksional[18].

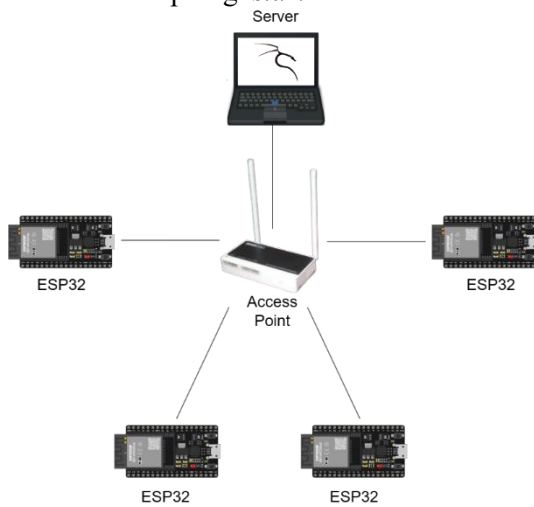
2.3 Perancangan Sistem

Adapun perancangan arsitektur sistem, termasuk perangkat keras dan perangkat lunak. Perancangan perangkat keras berfokus pada implementasi ESP32, perancangan perangkat lunak berfokus pada pengimplementasian kode serangan dalam ESP32.

2.3.1 Perancangan Arsitektur Sistem

Adapun perancangan arsitektur sistem yaitu perancangan perangkat keras dan perangkat lunak. Perancangan perangkat keras meliputi perancangan ESP32, Laptop yang

dijadikan *server*. Perancangan perangkat keras menggunakan topologi *star*. Topologi dapat dilihat pada Gambar 5, *client* atau ESP32 akan mengirim data ke *server*. *Server* hanya sebagai penerima data dari *client*, bukan *attacker*. *Attacker* mengganggu komunikasi dengan mengirim paket ICMP *Echo Request* secara intens dan mengirim paket ARP palsu dengan kecepatan 10 paket perdetik. Pada penelitian ini ESP32 *attacker* beroperasi mandiri setelah di program menggunakan PlatformIO. Gambar 5 merupakan topologi yang digunakan membentuk topologi *star*.



Gambar 5. Rancangan Topologi Star

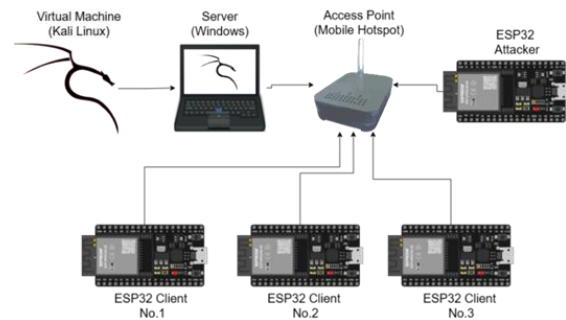
2.3.2 Perancangan Perangkat Keras

Adapun perancangan perangkat keras dalam membangun arsitektur sistem ini sebagai berikut:

1. Perancangan Perangkat ESP32 sebagai penyerang.
2. Perancangan Perangkat ESP32 sebagai *client*.
3. Perancangan *Server* dalam *Virtual Machine*.
4. Perancangan Wi-Fi Adapter Monitor Mode pada *Server*.
5. Perancangan *Access Point* sebagai Penghubung antar Perangkat.

2.3.3 Implementasi Perangkat Keras

Implementasi perangkat keras dilakukan sesuai dengan arsitektur sistem yang telah dirancang. Sistem ini terdiri dari beberapa komponen utama yang saling terhubung dalam satu jaringan lokal. Pada bagian ini merupakan tahap implementasi perangkat keras, sesuai dengan rancangan arsitektur pada Gambar 6.



Gambar 6. Rancangan Arsitektur Sistem

Keempat perangkat ESP32 dirangkai membentuk topologi jaringan *star*, *access point* menjadi pusat koneksi ESP32. Satu unit perangkat ESP32 diprogram untuk bertindak sebagai penyerang dari salah satu *client*. Wi-Fi adapter dihubungkan pada *virtual machine*, menggunakan sistem operasi kali linux mengubah mode *interface* dari *managed* menjadi *monitor mode*. Hasil penerapan dari perangkat keras yang dibuat sesuai dengan rancangan pada Gambar 6, dapat dilihat pada Gambar 7.



Gambar 7. Implementasi Arsitektur Sistem

2.3.4 Perancangan Perangkat Lunak

Adapun perancangan perangkat lunak dalam membangun arsitektur sistem ini sebagai berikut:

1. Perancangan Sistem Komunikasi Antara ESP32 *client* dengan *server* menggunakan protokol WebSocket.
2. Perancangan Serangan DoS pada ESP32 penyerang.
3. Perancangan Serangan ARP *Spoofing* pada ESP32 Penyerang.
4. Perancangan *Server* Menggunakan Kali Linux di dalam *Virtual Machine*.

2.3.5 Implementasi Perangkat Lunak

Implementasi perangkat lunak merupakan tahap realisasi dari perancangan sistem. Kode program dikembangkan untuk setiap komponen fungsional ESP32 *client*,

ESP32 *attacker*, dan *server*. Pada sisi *client*, perangkat lunak diprogram untuk mengumpulkan data metrik performa dan mengirimkannya ke *server* melalui protokol WebSocket. Sementara itu, perangkat lunak pada ESP32 penyerang diimplementasikan untuk menjalankan dua skenario serangan, yaitu ICMP *Flood* dan ARP *Spoofing*, dengan jeda pengiriman paket 100 ms artinya kecepatan mengirim paket sebesar 10 paket per detik. Di sisi *server*, implementasi meliputi pembuatan endpoint menggunakan Node.js untuk menerima data *client*, serta proses untuk menangkap dan menyimpan lalu lintas jaringan ke dalam *database* SQLite untuk dianalisis.

2.3.6 Implementasi Perangkat Lunak ESP32 Client

Pada bagian perangkat lunak ESP32 *client*, terdapat tiga proses yaitu koneksi ESP32 ke *access point*, koneksi ke *server* kemudian ESP32 mengirim data metrik ke *server*. Tampilan serial monitor pada *client* target dapat dilihat pada Gambar 8.

```
I (1843) main_task: Returned from app_main()
I (1843) wifi:cb:ada:idx:0 (ifx:0, 40:ee:15:68:07:10), tid:0, ssn:4, winSize:64
I (2283) WS_CLIENT: WEBSOCKET_EVENT_CONNECTED
I (2343) ARP_MONITOR: MAC Gateway kembali valid: 40:EE:15:68:07:10
I (5803) websocket_client: Started
I (5843) WS_CLIENT: WEBSOCKET_EVENT_CONNECTED
I (6363) WS_CLIENT: ACK received for seq: 0
I (6803) perfmon: Core 0 at 19%
I (6803) perfmon: Core 1 at 15%
I (7423) WS_CLIENT: ACK received for seq: 1
I (8393) WS_CLIENT: ACK received for seq: 2
I (9413) WS_CLIENT: ACK received for seq: 3
I (10593) WS_CLIENT: ACK received for seq: 4
```

Gambar 8. Komunikasi Perangkat *Client* Ke *Server*

2.3.7 Implementasi Perangkat Lunak ESP32 Attacker

Pada bagian perangkat lunak ESP32 *attacker*, terdapat 2 proses yaitu koneksi ESP32 ke *access point* dan mengirim 2 jenis paket serangan ke target. ESP32 penyerang memiliki tugas untuk menyerang target menggunakan serangan ICMP *flood* dan serangan ARP *spoofing*. Pada kedua serangan ini, masing masing paket deprogram memiliki jeda 100 ms antar paket. Untuk proses serangan dari perspektif penyerang atau serial monitor penyerang dapat dilihat pada Gambar 9.

```
I (1743) esp_netif_handlers: sta ip: 192.168.126.6, mask: 255.255.255.0, gw: 192.168.126.1
I (1743) WIFI_STATION: Mendapat IP:192.168.126.6
I (1743) MAIN_APP: WiFi Terhubung! Memulai semua task.
I (1743) ICMP_ATTACK: Memulai ICMP Flood ke 192.168.126.3
I (1753) ICMP_ATTACK: ICMP #1 terkirim (40 bytes)
I (1753) ARP_ATTACK: Memulai ARP spoofing dua arah (Mith)...
I (1763) main_task: Returned from app_main()
I (1853) ICMP_ATTACK: ICMP #2 terkirim (40 bytes)
I (1863) wifi:cb:ada:idx:0 (ifx:0, 40:ee:15:68:07:10), tid:0, ssn:4, winSize:64
I (1953) ICMP_ATTACK: ICMP #3 terkirim (40 bytes)
I (2053) ICMP_ATTACK: ICMP #4 terkirim (40 bytes)
I (2153) ICMP_ATTACK: ICMP #5 terkirim (40 bytes)
I (2253) ICMP_ATTACK: ICMP #6 terkirim (40 bytes)
I (2353) ICMP_ATTACK: ICMP #7 terkirim (40 bytes)
```

Gambar 9. ESP32 Mengirim Paket ICMP *Echo Request* dan ARP *Reply*

2.3.8 Implementasi Server

Server diimplementasikan dalam *virtual machine* dengan sistem operasi Kali linux pada laptop, di mana aplikasi *server* Node.js berbasis protokol WebSocket berjalan untuk menerima, memproses, menyimpan, dan menampilkan data yang dikirim oleh *client*. *Server* berjalan selama 30 menit. Saat *server* dimulai, *child process* akan dieksekusi untuk menjalankan *tcpdump*. *Tcpdump* dikonfigurasi untuk menangkap semua lalu lintas jaringan yang relevan dan menyimpannya ke dalam file berformat .pcap. File ini sangat penting untuk analisis pasca-pengujian, penangkapan paket menggunakan tools *Tcpdump* dengan filter paket yang berasal dari *client* dan *attacker*, dan jenis protokol paketnya ICMP *Echo Request* atau ARP *Reply*. Berikut adalah inisiasi server menggunakan perintah Node.js dapat dilihat pada Gambar 10.

```
(student@cyberates-lab) [-] /Downloads/websocket-server
$ sudo node server.js
Server utama berjalan di http://localhost:8765
Total durasi eksperimen: 30 menit.

[Memulai Sesi Eksperimen]
[Config] Nama file basis sesi ini: gabungan_ls_2025-07-15T07-16-01
[Capture] Menggunakan filter: (wlan host cc:db:a7:2e:7f:e8 or wlan host cc:db:a7:2e:99:08 or wlan host cc:db:a7:2d:73:c8 or wlan host 38:ae:a4:f2:3f:f0) or arp or icmp
[Capture] Memulai tcpdump di 'wlan0mon'
[Capture] Memulai tcpdump di 'eth0'
[Timer] Sesi ini akan berjalan selama 30 menit.
[tcpdump-wlan0mon stderr]: tcpdump: listening on wlan0mon, link-type IEEE802.11_RADI
0 (802.11 plus radiotap header), snapshot length 262144 bytes

[tcpdump-eth0 stderr]: tcpdump: listening on eth0, link-type EN10MB (Ethernet), snap
shot length 262144 bytes
```

Gambar 10. Inisiasi *Server* menggunakan Node.js

Sebelum pengambilan data, perangkat Wi-Fi *adapter* diubah menjadi *monitor* mode. Pada saat pengambilan data, menggunakan dua interface, setiap data aktivitas jaringan ditangkap menggunakan filter sesuai dengan IP dari setiap *client*. Pada gambar menunjukkan perubahan interface wlan0. Proses pengubahan dapat dilihat pada Gambar 11.

```
(student@cyberates-lab) [-]
$ sudo airmon-ng start wlan0
[sudo] password for student:

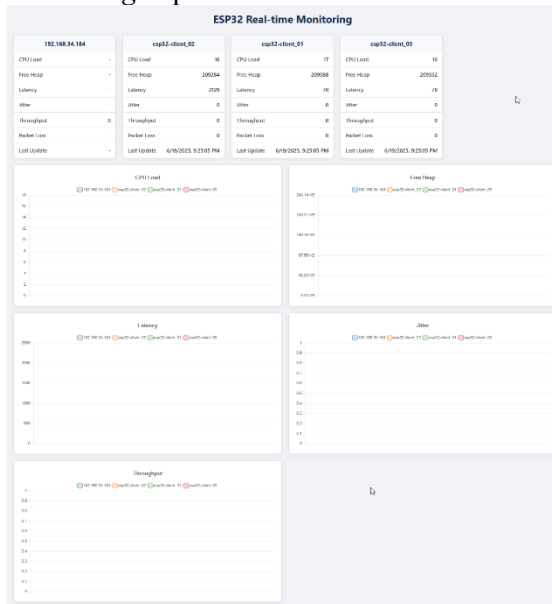
Found 3 processes that could cause trouble.
Kill them using 'airmon-ng check kill' before putting
the card in monitor mode, they will interfere by changing channels
and sometimes putting the interface back in managed mode

PID Name
832 dhclient
858 NetworkManager
1276 wpa_supplicant

PHY Interface Driver Chipset
phy0 wlan0 ath9k_htc Qualcomm Atheros Communications AR9271 982.
11n (mac80211 monitor mode vif enabled for [phy0]wlan0 on [phy0]wlan0mo
n)
(mac80211 station mode vif disabled for [phy0]wlan0)
```

Gambar 11. Mengubah Interface wlan0 menjadi monitor mode menggunakan Airmon-ng

Data yang diterima saat *server* berjalan dapat ditampilkan pada *dashboard monitoring*. *Dashboard monitoring* digunakan hanya untuk keperluan penelitian agar dapat melihat secara langsung dampak yang dihasilkan oleh ESP32 penyerang. Berikut tampilan *dashboard monitoring* dapat dilihat Gambar 12.



Gambar 12. Dashboard Monitoring ESP32 Client

Server menyimpan data menggunakan aplikasi SQLite yang telah disiapkan pada sisi *server* dengan modul better-sqlite3, *server* otomatis akan menerima data metrik yang berasal dari client dan disimpan ke database. Hasil database yang telah disimpan file .db dibuka dapat dilihat pada Gambar 13. Setelah 30 menit, *server* melakukan penggabungan dari file *packet capture* dari interface wlan0mon dan eth0 menjadi satu file .pcap menggunakan perintah Mergecap fitur dari bawaan Wireshark. Hal ini memudahkan pada saat pengolahan data setelah pengambilan data. Pada dua file packet capture tersebut akan di *decrypt* menggunakan Aircap-ng karena paket yang berasal dari wlan0mon adalah paket yang di enkapsulasi menggunakan protokol 802.11 atau masih dalam bentuk paket *encrypt*. Pada Gambar 13 menunjukkan bahwa data metrik tersimpan ke dalam database.

ip	hostname	metrics	timestamp
Filter	Filter	Filter	Filter
192.168.126.3	esp32-client_01	("tx_seq":0, "status":"starting")	2025-07-13 05:36:19
192.168.126.3	esp32-client_01	("hostname":"esp32-...")	2025-07-13 05:36:20
192.168.126.3	esp32-client_01	("hostname":"esp32-...")	2025-07-13 05:36:21
192.168.126.4	esp32-client_02	("tx_seq":0, "status":"starting")	2025-07-13 05:36:22
192.168.126.3	esp32-client_01	("hostname":"esp32-...")	2025-07-13 05:36:22
192.168.126.3	esp32-client_01	("hostname":"esp32-...")	2025-07-13 05:36:23
192.168.126.4	esp32-client_02	("hostname":"esp32-...")	2025-07-13 05:36:23
192.168.126.3	esp32-client_01	("hostname":"esp32-...")	2025-07-13 05:36:24

Gambar 13. Data Metrik yang Tersimpan pada Database

2.4 Implementasi Sistem

Implementasi dilakukan sesuai dengan arsitektur sistem yang telah dirancang. Rencananya adalah mengimplementasikan server dan data metrik performa yang dapat disimpan kedalam database, pergerakan lalu lintas data antara ESP32 penyerang dan *client* dapat dianalisis mulai dari *latency*, *throughput*, *jitter*, dan *packet loss* melalui *server* yang telah diimplementasikan. Pada sisi penyerang akan dilakukan implementasi serangan ke *client* target sesuai dengan perancangan serangan yaitu ESP32 penyerang akan melancarkan serangan ke salah satu perangkat ESP32 *client*.

2.5 Pengujian Sistem

Pada pengujian dilakukan untuk mengetahui proses dan dampak dari serangan ICMP Flood dan ARP spoofing yang dilakukan oleh perangkat ESP32 attacker terhadap ESP32 client dalam jaringan WSN dengan topologi star. Setelah melakukan pengujian, diperoleh data hasil pengujian saat melakukan pengimplementasian serangan ICMP Flood dan ARP Spoofing dalam jaringan WSN berbasis topologi star. Data yang dihasilkan mencakup parameter Quality of Service (QoS), seperti *throughput*, *latency*, *jitter*, dan *packet loss*. Selain itu, data pada beban CPU akan digunakan untuk mengetahui kinerja dari ESP32 client, apakah dari penyerangan melakukan ESP32 dapat membuat ESP32 client disconnect atau down. Untuk pengambilan data dilakukan selama 30 menit untuk setiap interval pengiriman data client, yaitu 1 detik, 5 detik, 10 detik.

Berikut adalah perhitungan hasil dari setiap parameter QoS mulai dari *throughput*, *latency*, *jitter*, dan *packet loss*. QoS diukur karena ingin mengetahui seberapa besar dampak yang dihasilkan dari satu perangkat ESP32.

1. Throughput

Throughput merupakan jumlah total kedatangan paket yang sukses yang diamati pada tujuan [19]. Untuk menghitung *throughput* dapat menggunakan Persamaan 1 [19].

$$\text{Throughput} = \frac{\text{paket yang diterima}}{\text{lama pengiriman}} \quad (1)$$

2. Latency

Latency merupakan waktu tunda suatu paket yang diakibatkan oleh proses

transmisi dari satu titik ke titik lain yang menjadi tujuannya[20]. Persamaan perhitungan *latency* dapat menggunakan Persamaan 2[20].

$$Latency = \text{Waktu kirim} - \text{Waktu terima} \quad (2)$$

3. Jitter

Nilai *jitter* adalah pergeseran waktu antar paket yang terjadi pada jaringan IP. Besarnya nilai *jitter* sangat dipengaruhi oleh variasi beban trafik dan tumbukan antar paket, juga dikenal sebagai *congestion*[8]. Perhitungan *jitter* dapat menggunakan Persamaan 3[21].

$$Jitter = \frac{\text{Total variasi latency}}{\text{Total paket diterima}} \quad (3)$$

$$\text{Variasi latency} = (\text{Latency1} - \text{Latency2} - \text{Latency(n-1)}) \quad (4)$$

4. Packet Loss

Packet loss adalah kegagalan transmisi paket IP mencapai tujuannya[20]. Perhitungan *packet loss* dapat menggunakan Persamaan 4[21].

$$PL = \frac{(\text{paket data dikirim} - \text{paket data diterima})}{\text{paket data dikirim}} \times 100\% \quad (5)$$

2.6 Kesimpulan dan Saran

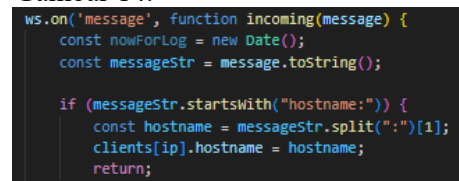
Tahap akhir dari penelitian ini adalah menarik kesimpulan berdasarkan hasil pengujian sistem dan data yang telah terkumpul. Berdasarkan temuan-temuan ini, saran-saran akan diberikan untuk perbaikan lebih lanjut dan pengembangan di masa mendatang.

3. HASIL DAN PEMBAHASAN

Pada bagian ini membahas dari temuan dari awal penelitian hingga akhir. Kegagalan pengiriman data ini terjadi sehingga metrik *Quality of Service* (QoS) lainnya seperti *latency* dan *jitter* menjadi tidak dapat diukur. Fenomena ini mengkonfirmasi bahwa dengan meracuni ARP *cache* karena pada saat *client* mengirim paket *request* menggunakan protokol WebSocket, *server* seharusnya membalas dengan paket *handshake*, setelah itu *handshake* ini diterima oleh *client*, terjadilah koneksi WebSocket namun dampak pada serangan ARP *spoofing* ini, mengganggu proses tersebut, yaitu paket *request* tidak sampai ke *server* karena *client* mencoba berkomunikasi dengan penyerang namun penyerang tidak menyediakan mekanisme protokol WebSocket, sehingga paket *request* ke penyerang tidak

ditindak atau diteruskan atau paket *request* tidak terkirim sama sekali. Hal ini menunjukkan bahwa satu perangkat ESP32 penyerang mampu mengisolasi perangkat lain dari jaringan menggunakan ARP *reply* palsu yang menyebabkan kondisi *denial of service* tanpa perlu mengerahkan sumber daya komputasi yang besar. Adapun kemungkinan penyerangan terjadi ketika menggunakan ESP32 agar dapat mengubah data dalam protokol WebSocket yaitu:

1. Menginput data palsu dari sisi kerentanan program *server* (duplikasi *hostname* pada *client* walaupun ip *client* dan penyerang berbeda), sehingga penyerang dengan *hostname* yang sama dengan *client* akan mengirim data yang berasal dari *client* dan penyerang. Contoh kode program *server* yang rentan dapat dilihat pada Gambar 14.



Gambar 14. Kode program pada server yang rentan.

2. ESP32 penyerang melakukan duplikasi Wi-Fi asli dengan Wi-Fi palsu sekaligus menjadi *client* palsu. Celah ini dapat dilakukan ketika *client* asli tidak ada mekanisme untuk memeriksa bahwa *client* terhubung pada Wi-Fi asli atau palsu. Tujuan duplikasi ini memaksa *client* terhubung dengan AP palsu dengan mengirimkan paket *deauthentication* ke *client* asli atau memaksa *client* asli untuk *restart* yang mengakibatkan koneksi terputus dengan Wi-Fi asli sehingga memungkinkan *client* asli akan terhubung ke *client* palsu. Akibatnya didalam database akan tercatat data yang berasal dari *client* namun alamat IP yang terhubung pada *server* adalah *client* palsu. ESP32 penyerang belum dapat memanipulasi data namun, penyerang dapat mengirimkan data palsu ke *server*. Sehingga membuat *server* akan menerima data palsu dan data asli dengan alamat IP yang sama. Kemungkinan untuk memanipulasi data secara *realtime*, penyerang harus dapat menebak paket dengan nomor urut yang benar pada saat

pengiriman data dari *client* asli ke *server* berlangsung. Pada gambar ini memperlihatkan bahwa *client* mengirim data ke *server* melalui IP penyerang, dan di *server* tetap menerima data palsu tersebut karena *server* hanya memisahkan 3 jenis yaitu (IP, *Hostname*, dan *Payload*) apapun *payload* yang dikirimkan selama penyerang memiliki *hostname* dan *payload* maka *server* akan mencatat ke dalam tabel *database*. Serangan ini tidak termasuk ke dalam ARP *spoofing* karena ESP32 penyerang tidak perlu mengirim paket ARP *reply* palsu ke *server* maupun ke *client*. ESP32 penyerang hanya melakukan duplikasi jaringan Wi-Fi dan menjadi *client* palsu yang terhubung ke *server*. Hasil *Database* yang telah terkontaminasi oleh penyerang dapat dilihat pada Gambar 15.

id	ip	hostname	metrics	timestamp
13	192.168.137.101	esp32-client_01	{"hostname":"esp32-client_01","cpu_load":...	2025-08-01 16:14:55
14	192.168.137.101	esp32-client_01	{"hostname":"esp32-client_01","cpu_load":...	2025-08-01 16:14:56
15	192.168.137.101	esp32-client_01	{"action":"transfer_funds","from":"victim_...	2025-08-01 16:14:56
16	192.168.137.101	esp32-client_01	{"hostname":"esp32-client_01","cpu_load":...	2025-08-01 16:14:57

Gambar 15. *Database* yang telah terkontaminasi oleh penyerang.

3.1 Dampak Serangan terhadap QoS

Pada pengujian sistem, bagian ini membahas penelitian yang dilakukan dan setelah mengimplementasikan serangan, menyajikan data dampak dari serangan terhadap QoS.

3.3.1 Throughput

Pada Gambar 16 terlihat bahwa pada serangan arp-spoof dan gabungan, *client* target tidak dapat terhubung ke *server*, namun pada serangan icmp-flood, *throughput* tertinggi yaitu pada 1 detik interval pengiriman data *client* sebesar 35 kbps dibandingkan dengan *client* non-target sebesar rata-rata 20,4 kbps. Hal ini menunjukkan bahwa ESP32 *client* target merespon paket ICMP *echo request* lebih sering sehingga meningkatkan *throughput* pada *client* target.



Gambar 16. Grafik *Throughput* Pada ESP32 *Client*

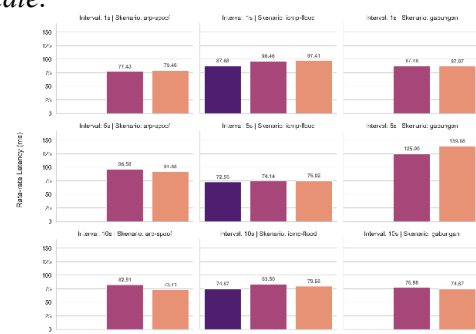
Tabel 1. *Throughput* pada ESP32 *Client* Interval 1 Detik

Interval	Skenario	Hostname	Throughput (kbps)
1 detik	Arp-spoof	Client01	0 kbps
		Client02	12.69 kbps
		Client03	12.28 kbps
	Icmp-flood	Client01	35.06 kbps
		Client02	20.45 kbps
		Client03	20.44 kbps
	Gabungan	Client01	0 kbps
		Client02	12.35 kbps
		Client03	12.17 kbps

Pada Tabel 1 terlihat *throughput* Client01 bernilai 0 karena pada serangan arp-spoof menyebabkan Client01 tidak terhubung ke *server*.

3.3.2 Latency

Pada Gambar 17 terlihat bahwa pada serangan arp-spoof dan gabungan, *client* target tidak dapat terhubung ke *server*, namun pada serangan icmp-flood, *client* target memiliki *latency* lebih rendah daripada yang *client* non-target yaitu salah satunya pada serangan intensitas tinggi icmp-flood interval 1 detik sebesar 87,68 ms, sedangkan pada *client* kedua sebesar 96,46 ms. Grafik dapat dilihat pada Gambar 16, hal ini kemungkinan terjadi karena kondisi CPU *client* target selalu aktif, dan tidak sempat mengalami kondisi *idle*.



Gambar 17. Grafik *Latency* pada ESP32 *Client*

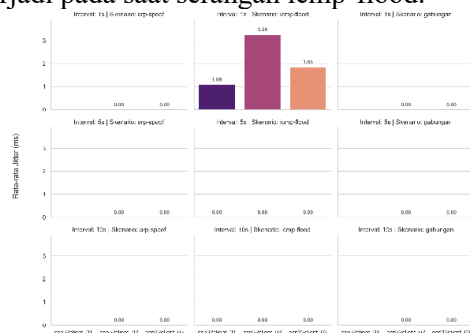
Tabel 2. *Latency* pada ESP32 Client Interval 1 Detik

Interval	Skenario	Hostname	Latency (ms)
1 detik	Arp-spoof	Client01	-
		Client02	77.43 ms
		Client03	79.46 ms
	Icmp-flood	Client01	87.68 ms
		Client02	96.46 ms
		Client03	97.41 ms
	Gabungan	Client01	-
		Client02	87.48 ms
		Client03	87.07 ms

Pada Tabel 2 terlihat *latency* Client01 tidak dapat terukur karena serangan arp-spoof menyebabkan Client01 tidak terhubung ke *server*.

3.3.3 Jitter

Pada Gambar 18 terlihat bahwa pada serangan arp-spoof dan gabungan, client target tidak dapat terhubung ke *server*, namun pada serangan icmp-flood dengan interval pengiriman data client 1 detik, yaitu pada *client* target memiliki *jitter* 1,09 ms kemungkinan ada hubungannya dengan *latency*. Hal ini kemungkinan terjadi kondisi CPU pada *client* target menjadi responsif, sehingga membuat pengiriman data lebih stabil. Terlihat pada Gambar 18 bahwa *jitter* terjadi pada saat serangan icmp-flood.



Gambar 18. Grafik *Jitter* pada ESP32 Client

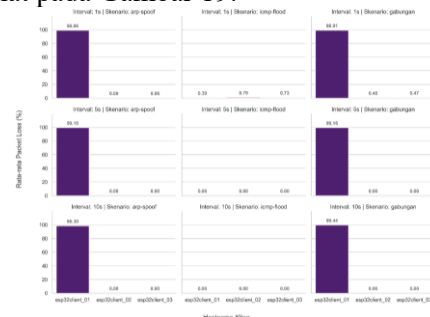
Tabel 3. *Jitter* pada ESP32 Client Interval 1 Detik

Interval	Skenario	Hostname	Jitter (ms)
1 detik	Arp-spoof	Client01	-
		Client02	0 ms
		Client03	0 ms
	Icmp-flood	Client01	1.09 ms
		Client02	3.26 ms
		Client03	1.85 ms
	Gabungan	Client01	-
		Client02	0 ms
		Client03	0 ms

Pada Tabel 3 terlihat *jitter* pada serangan icmp-flood menyebabkan penambahan beban pada jaringan, untuk serangan arp-spoof dan gabungan, Client02 dan Client03 tidak mengalami penurunan kualitas pada transmisi data, sementara untuk Client01 tidak terukur karena tidak terhubung ke *server*. Pada serangan gabungan perangkat penyerang menjadi kurang efektif karena harus mengeksekusi 2 serangan sekaligus.

3.3.4 Packet Loss

Pada Gambar 19 terlihat bahwa pada serangan arp-spoof dan gabungan untuk seluruh interval, *client* target tidak dapat terhubung ke *server*, ini penyebab nilai packet loss mendekati 100 %, karena dampak dari serangan arp-spoof menggunakan ESP32 attacker membuat komunikasi *server* dengan ESP32 *client* target terganggu. Grafik dapat dilihat pada Gambar 19.



Gambar 19. Grafik Packet Loss pada ESP32 Client

Tabel 4. Packet Loss pada ESP32 Client

Interval	Skenario	Hostname	Packet Loss (%)
1 detik	Arp-spoof	Client01	98.86 %
		Client02	0 %
		Client03	0.06 %
	Icmp-flood	Client01	0.39 %
		Client02	0.79 %
		Client03	0.73 %
	Gabungan	Client01	98.91 %
		Client02	0.4 %
		Client03	0.47 %

3.2 Dampak Serangan terhadap Kinerja Perangkat

Pada bagian ini selain menyajikan data dampak serangan terhadap QoS, kinerja perangkat dalam ESP32 dapat disajikan dalam bentuk grafik untuk melihat efektivitas dari serangan menggunakan ESP32, yaitu dari CPU *load* dan penggunaan memori.

3.4.1 CPU Load

Pada serangan icmp-flood dengan semua interval, *client* target memiliki CPU load tertinggi daripada *client* non-target. Salah satunya pada waktu interval pengiriman data *client* 1 detik, *client* target memiliki nilai sebesar 17,63 % sedangkan untuk *client* non target memiliki nilai sebesar 17,09 %. Hal ini menunjukkan bahwa serangan icmp-flood menggunakan satu perangkat ESP32 tidak cukup untuk membuat satu perangkat ESP32 lain *down* atau *overload*. Grafik dapat dilihat pada Gambar 20.



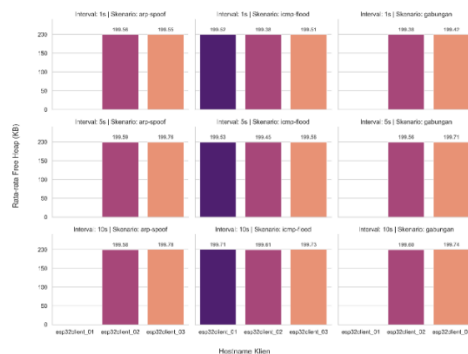
Gambar 20. Grafik CPU Load pada ESP32 Client

Tabel 5. CPU Load pada ESP32 Client Interval 1 Detik

Interval	Skenario	Hostname	CPU load (%)
1 detik	Arp-spoof	Client01	-
		Client02	16.86 %
		Client03	16.9 %
	Icmp-flood	Client01	17.63 %
		Client02	17.09 %
		Client03	17.1 %
	Gabungan	Client01	-
		Client02	16.68 %
		Client03	16.82 %

3.4.2 Free Heap Memory

Pada pengukuran penggunaan memori ESP32 yaitu *Free Heap Memory*, atau memori dinamis yang tersedia pada ESP32. Pada pengujian dampak, untuk satu perangkat ESP32 tidak cukup untuk menurunkan memori dinamis pada ESP32. Hal ini kemungkinan terjadi perangkat penyerang tidak cukup banyak untuk mengirim paket ICMP *echo request* kepada *client* target. Pada *Free Heap Memory* dari seluruh perangkat ESP32 berada di kisaran 199 KB. Grafik dapat dilihat pada Gambar 21.



Gambar 21. Grafik Free Heap Memory pada ESP32 Client

Tabel 6. Free Heap Memory pada ESP32 Client Interval 1 Detik

Interval	Skenario	Hostname	Free Heap Memory (KB)
1 detik	Arp-spoof	Client01	-
		Client02	199.56 KB
		Client03	199.55 KB
	Icmp-flood	Client01	199.52 KB
		Client02	199.38 KB
		Client03	199.51 KB
	Gabungan	Client01	-
		Client02	199.38 KB
		Client03	199.42 KB

Pada Tabel 6 *Free Heap Memory* Client01 tidak terukur akibat serangan arp-spoof Client01 tidak dapat terhubung ke server .

3.3 Analisis Kapabilitas ESP32 sebagai Penyerang

Pada bagian ini menjelaskan kapabilitas untuk satu perangkat ESP32 penyerang digunakan untuk mengirim paket ICMP *echo request* dan paket ARP *reply* palsu. Ukuran dari paket ICMP *echo request* sebesar 74 Bytes dan Ukuran paket ARP *reply* sebesar 118 Bytes. Terlihat pada serangan arp-spoof interval 1 detik dengan total paket yang dikirim sebesar 17597 didapatkan rata-rata paket perdetik sebesar 9,78 PPS, ini membuktikan bahwa ESP32 konsisten mengirim paket ARP *reply* palsu sesuai dengan jeda antar paket pada program yaitu 100 ms, itu artinya ESP32 harus mengirim paket serangan 10 paket per detik. Pada serangan icmp-flood interval 1 detik, ESP32 dapat konsisten mengirim rata rata paket perdetik yaitu sebesar 9,87 PPS. Data selengkapnya dapat dilihat pada Tabel 7.

Tabel 7. Kapabilitas Perangkat ESP32 sebagai Penyerang

Skenario	Interval	Rata-rata (PPS)	Throughput (kbps)
Arp-spoof	1 detik	9.78 PPS	9.23 kbps
	5 detik	9.98 PPS	9.42 kbps
	10 detik	9.67 PPS	9.13 kbps
Icmp-flood	1 detik	9.87 PPS	5.85 kbps
	5 detik	9.91 PPS	5.87 kbps
	10 detik	9.90 PPS	5.86 kbps
Gabungan	1 detik	19.91 PPS	15.33 kbps
	5 detik	19.62 PPS	15.06 kbps
	10 detik	19.76 PPS	15.25 kbps

Pada Tabel 7 kapabilitas untuk 1 perangkat ESP32 penyerang sesuai dengan jeda program yaitu setiap 1 serangan selama 100 ms yang artinya dalam 1 detik terdapat 10 paket per detik.

4. KESIMPULAN

Pada penelitian ini dapat ditarik beberapa kesimpulan. Mekanisme serangan DoS berbasis ICMP Flood dan ARP Spoofing dari perangkat ESP32 berhasil diimplementasikan, dimana setelah program ditanamkan secara permanen, ESP32 dapat bekerja otonom. Mekanisme serangan Denial of Service (DoS) berbasis ICMP Flood dan serangan ARP Spoofing dari perangkat ESP32 dapat diimplementasikan dengan memprogram perangkat menggunakan framework ESP-IDF untuk mengirimkan paket ICMP Echo Request secara terus-menerus ke alamat IP target dan mengirimkan paket ARP reply palsu ke client target dan server. Namun serangan ICMP Flood dan ARP Spoofing dapat dilakukan dengan diawali pengumpulan informasi target yang akan diserang menggunakan bantuan perangkat lain yang memiliki aksesibilitas kontrol yang lebih kuat seperti laptop. Setelah menanamkan program secara permanen, perangkat ESP32 dapat bekerja secara otonom saat mengeksekusi perintah program. Namun untuk memanipulasi data melalui ARP Spoofing tidak tercapai. Hal ini disebabkan karena protokol WebSocket membutuhkan proses handshake dan sinkronisasi sequence number antar paket. Pada saat ARP Spoofing diterapkan, ESP32 client tidak dapat melakukan handshake dengan server sehingga koneksi WebSocket tidak pernah terbentuk dan data tidak dimanipulasi. Penelitian ini juga menemukan kemungkinan serangan lain yang tidak melibatkan ARP Spoofing, yaitu dengan memanfaatkan kerentanan server berupa duplikasi hostname

dan teknik duplikasi Wi-Fi. Melalui metode ini, penyerang dapat menjadi client palsu dan mengirimkan data palsu ke server, sehingga database dapat terkontaminasi meskipun data belum dapat dimanipulasi secara realtime. Dampak pada QoS paling berisiko pada serangan ARP spoofing, di mana throughput menjadi nol, sementara ICMP Flood hanya sedikit menurunkan stabilitas jaringan. Penelitian ini menunjukkan bahwa untuk satu perangkat ESP32 memiliki dampak yang kurang efektif khususnya pada ICMP Flood karena paket ICMP dikirimkan dengan kecepatan 10 paket per detik. Hal ini untuk menjaga kestabilan dari perangkat penyerang agar perangkat tidak mengalami restart, dan menunjukkan bahwa perangkat ESP32 dapat difungsikan sebagai platform serangan siber yang efektif pada jaringan yang menggunakan protokol WebSocket meskipun memiliki keterbatasan aksesibilitas kontrol.

DAFTAR PUSTAKA

- [1] Agus. Kurniawan, *Internet of Things projects with ESP32 : build exciting and powerful IoT projects using the all-new Espressif ESP32*. Packt Publishing, 2019.
- [2] D. Andaraweera, H. Kottegoda, P. Paliheena, T. Thilakarathne, and A. D. H, "Security Challenges and Solutions in Internet of Things (IoT): A Review," Nov. 2022. [Online]. Available: <https://www.researchgate.net/publication/365349394>
- [3] A. Alsaaidah, O. Almomani, A. A. Abu-Shareha, M. M. Abualhaj, and A. Achuthan, "ARP Spoofing Attack Detection Model in IoT Networks Using Machine Learning: Complexity vs. Accuracy," *Journal of Applied Data Sciences*, vol. 5, no. 4, pp. 1850–1860, 2024, doi: 10.47738/jads.v5i4.374.
- [4] F. Kpieleh, "A Review of Ddos Attack Detection in Iot Networks," *Advances in Multidisciplinary and scientific Research Journal Publication*, vol. 10, no. 4, pp. 43–54, Nov. 2022, doi: 10.22624/AIMS/DIGITAL/V10N4P6.
- [5] S. M. Morsy and D. Nashat, "D-ARP: An Efficient Scheme to Detect and Prevent ARP Spoofing," *IEEE Access*, vol. 10, pp. 49142–49153, 2022, doi: 10.1109/ACCESS.2022.3172329.
- [6] S. Friedl and G. Pernul, "Forensic Analysis of an IoT ARP Spoofing Attack," in *12th International Symposium on Digital*



- Forensics and Security, ISDFS 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/ISDFS60797.2024.10527302.
- [7] M. A. Burhanuddin, A. Abdul-Jabbar Mohammed, R. Ismail, M. Emad Hameed, A. Noori Kareem, and H. Basiron, "A Review on Security Challenges and Features in Wireless Sensor Networks: IoT Perspective," *Journal of Telecommunication, Electronic and Computer Engineering*, vol. 10, 2018.
- [8] A. Budiman, M. Ficky Duskarnaen, and H. Ajie, "Analisis Quality Of Service (QoS) pada Jaringan Internet SMK Negeri 7 Jakarta," vol. 4, 2020.
- [9] I. Kravets, "What is PlatformIO." Accessed: Jul. 07, 2025. [Online]. Available: <https://github.com/platformio/platformio-docs/blob/develop/what-is-platformio.rst>
- [10] Espressif Systems, "ESP-IDF Programming Guide." Accessed: Jul. 07, 2025. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>
- [11] D. Williamson and R. O'Reilly, "WebTransport and WebSockets: An Empirical Analysis of Connection Time, Message Response, and Payload Efficiency," in *2023 34th Irish Signals and Systems Conference (ISSC)*, 2023, pp. 1–6. doi: 10.1109/ISSC59246.2023.10162060.
- [12] H. Kamaludin, M. Y. Jamal, N. H. A. Rahman, N. Z. M. Safar, and S. A. Ishak, "Implementing Virtual Machine: A Performance Evaluation," in *Advances in Intelligent Systems and Computing*, Springer, 2020, pp. 373–381. doi: 10.1007/978-3-030-36056-6_35.
- [13] R. R. Asaad, "Penetration Testing: Wireless Network Attacks Method on Kali Linux OS," *Academic Journal of Nawroz University*, vol. 10, no. 1, pp. 7–12, Feb. 2021, doi: 10.25007/ajnu.v10n1a998.
- [14] S. Zaim, "Apakah WPA/WPA2 Benar-benar Aman? Dekripsi Paket Data Terenkripsi pada WPA/WPA2," *Semnasif: Seminar Nasional Informatika*, vol. 1, no. 1, Informatika Dalam Pengelolaan Sumber Daya Alam, Nov. 2015.
- [15] V. Kapustin and N. Paulauskas, "Analysis Of TCP Flood Attack Using Netflow," *Mokslas - Lietuvos ateitis*, vol. 15, no. 0, pp. 1–7, Jul. 2023, doi: 10.3846/mla.2023.18847.
- [16] Node.js Foundation, "About Node.js." Accessed: Jul. 07, 2025. [Online]. Available: <https://nodejs.org/en/about>
- [17] R. Tri Novita, I. Gunawan, I. Marleni, O. Gregarius Grasia, and M. Nanda Valentika, "Analisis Keamanan Wifi Menggunakan Wireshark," *JES: Jurnal Elektro Smart*, vol. 1, no. 1, Aug. 2021, Accessed: Dec. 09, 2023. [Online]. Available: <https://www.sttcepu.ac.id/jurnal/index.php/jes/article/download/159/95>
- [18] C. Hummert and D. Pawlaszczyk, *Mobile forensics - the file format handbook: Common file formats and file systems used in mobile devices*. Springer International Publishing, 2022. doi: 10.1007/978-3-030-98467-0.
- [19] M. M. Abidin, "Internet Network Analysis on Local Provider X Using QoS Method," *INTELLITHINGS: Journal of AI and the IoT*, vol. 1, no. 1, Jan. 2025, Accessed: Aug. 18, 2025. [Online]. Available: www.wireshark.org
- [20] A. A. Rachmat, S. Bahri, and H. Hasfani, "Optimasi Pengiriman Data Sensor menggunakan Pemrosesan Paralel Berbasis Internet of Things (IoT)," *Coding: Jurnal Komputer dan Aplikasi*, vol. 13, no. 1, pp. 45–54, 2025, doi: 10.26418.
- [21] H. Anom Susetyo Aji Nugroho, Sonhaji, and A. Chandra Prasetyo, "Evaluation of Student WiFi Network Performance: Throughput, Delay, Jitter, and Packet loss analysis," *Jurnal Ilmu Komputasi BATRISI: Bahari Teknik Informatika & Sistem Informasi*, vol. 8, no. 1, Maritime Journal of Informatics Engineering & Information Systems, pp. 23–27, Jul. 2024.

